RECIPROCAL GUIDANCE: ORCHESTRATING DRAFT AND VERIFY BUDGETS FOR ADVANCING THE DIFFUSION-AR SELF-SPECULATION FRONTIER

Linye Wei[1,2,*] **Shutian Zheng**[3,*] **Haoyu Zeng**[4] **Meng Li**[1,2,5,†]
[1]Institute for Artificial Intelligence, Peking University
[2]School of Integrated Circuits, Peking University
[3]College of Engineering, Peking University
[4]School of Airspace Science and Engineering, Shandong University
[5]Beijing Advanced Innovation Center for Integrated Circuits

arXiv:2609.34388v1 [cs.CL] 28 Sep 2026

ABSTRACT

Diffusion drafting with autoregressive (AR) verification has emerged as a promising paradigm for efficient speculative decoding. Recent self-speculation models, represented by Nemotron-Labs-Diffusion, further simplify the speculative pipeline by unifying drafting and verification within a shared backbone, while enabling longer acceptance lengths. However, the Pareto frontier between aggregate and per-request throughput remains underexplored. At low concurrency, sequential draft-verify execution requires two model forward passes per round, limiting the effective tokens per forward (TPF). By contrast, at high concurrency, longer drafts incur increasingly expensive computation, forcing individual requests to operate under constrained speculation budgets and preventing full exploitation of the full-backbone drafter. Our key observation indicates that drafting and verification exhibit reciprocal predictability. Draft logits can anticipate likely verification mismatches, while recent verification outcomes predict future drafting utility and suitable block sizes. Building on this observation, we introduce **Reciprocal Guidance** (**RecGuide**), a runtime draft-verify orchestration framework that adapts speculative decoding to varying serving loads. RecGuide exploits spare compute capacity through verification-overlapped drafting at low concurrency, while dynamically allocating request-specific draft block sizes as the workload becomes increasingly compute-intensive. Experiments across a wide range of concurrency levels demonstrate consistent throughput improvements over vanilla self-speculation, achieving up to 1.8× speedup.

1 INTRODUCTION

Speculative decoding (Chen et al., 2023; Leviathan et al., 2023) has emerged as an important technique for accelerating large language model (LLM) inference. It employs a lightweight draft model to autoregressively generate multiple candidate tokens, which are then verified and potentially accepted in a single forward pass. Recently, inspired by diffusion language models (dLLMs) (Nie et al., 2026; Ye et al., 2025a; Cheng et al., 2026a), diffusion-based speculative decoding (Chen et al., 2026; Huang et al., 2026; Cheng et al., 2026b) uses a predictor with bidirectional attention to generate draft tokens in parallel, followed by autoregressive verification. By reducing sequential drafting overhead while maintaining high acceptance rates, this paradigm has attracted growing attention.

Alongside this line of work, self-speculation methods such as Nemotron-Labs-Diffusion (Fu et al., 2026; Liu et al., 2026b) use a single shared backbone and switch between bidirectional and causal attention to support diffusion drafting and autoregressive verification without requiring a separate drafter. This design simplifies the speculative pipeline (Zhang et al., 2026b) and, by reusing the full target backbone itself as the drafter (Zhang et al., 2024), offers stronger task generalization and the potential for longer acceptance lengths than lightweight draft models.

*Equal contribution. †Corresponding author: meng.li@pku.edu.cn.

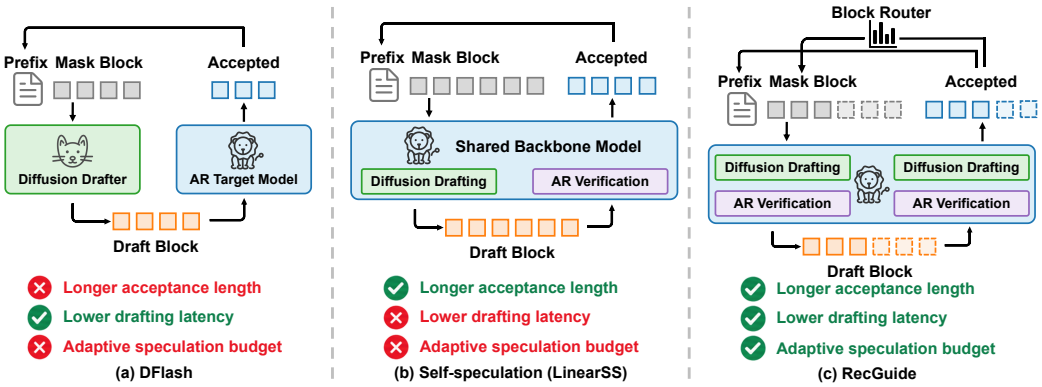


Figure 1: Comparison between RecGuide and existing diffusion-based speculative decoding.

However, concurrency varies dynamically in real-world serving workloads (Chen et al., 2024), while current self-speculation typically relies on fixed draft lengths (referred to as block sizes hereafter) tailored to different concurrency regimes. We argue that this coarse-grained allocation is insufficient (Sadhukhan et al., 2025). **At low concurrency**, the sequential draft-then-verify pipeline requires two expensive forward passes per speculative round, reducing the effective tokens per forward (TPF) to roughly half of the acceptance length and substantially diminishing the gains enabled by the full-backbone drafter. **At high concurrency**, by contrast, maintaining a large block size for every request becomes impractical, as the resulting computation pushes GPU execution into the compute-bound regime and significantly increases forward-pass latency. Yet choosing a small block size underutilizes self-speculation's potential for longer acceptance lengths compared with conventional speculative decoding paradigms. This motivates the following question:

Can we orchestrate draft and verify budgets to advance the aggregate and per-request throughput frontier, thereby making self-speculation a competitive paradigm for speculative decoding?

To address this challenge, we propose **Reciprocal Guidance** (**RecGuide**), grounded in our core observation that drafting and verification on the shared backbone are not independent, but exhibit strong reciprocal predictability. On the one hand, draft logits provide informative signals for anticipating likely mismatch positions in subsequent AR verification, as well as a likely correction token. On the other hand, recent verification outcomes reflect the local drafting difficulty of each request and provide an effective signal for selecting an appropriate block size in the next round.

Building on these observations, RecGuide provides a draft-verify orchestration framework for self-speculation with complementary strategies across varying concurrency levels, as illustrated in Figure 1. At low concurrency, RecGuide predicts the upcoming verification outcome from the draft probability distribution and co-batches likely next-round drafts with AR verification. This overlap is particularly natural for self-speculation because drafting and verification share the same weights, enabling simple co-batching without model switching or separate devices while exploiting spare compute capacity in the memory-bound regime (Liu et al., 2025; Kumar et al., 2026). As the workload increases, RecGuide instead uses verification feedback to estimate future drafting utility and dynamically assigns request-specific draft block sizes. Together with bucketed serving based on block size, this strategy balances the drafting capability of the model against available GPU compute.

We evaluate RecGuide under diverse serving regimes. Our method consistently improves throughput over vanilla self-speculation, enhancing speculative efficiency under light workloads and serving efficiency under heavy workloads. Our main contributions can be summarized as follows:

- We characterize the efficiency bottlenecks and pipeline challenges of self-speculation, and uncover reciprocal predictability between drafting and verification.
- We propose RecGuide, which orchestrates draft and verify budgets across varying serving loads through predictive draft overlap and adaptive block sizing.
- Extensive experiments demonstrate that RecGuide achieves consistent throughput improvements over vanilla self-speculation, with up to a 1.8× speedup.

2 RELATED WORK

Drafting Paradigms for Speculative Decoding. Speculative decoding (Chen et al., 2023; Leviathan et al., 2023; Cai et al., 2024; Zhong et al., 2024) has become a widely adopted approach for accelerating large language model inference. It uses a lightweight drafter to propose multiple future tokens, which are verified in parallel by the target model, enabling multiple tokens to be accepted per target-model forward without altering the generation distribution. Early methods relied on smaller standalone models sharing the target vocabulary, making speedup sensitive to drafter latency. EAGLE series (Li et al., 2024b;a; 2026b) improve drafting efficiency by exploiting target model representations. Nevertheless, autoregressive drafting still incurs latency that grows with draft length. Inspired by diffusion (Ho et al., 2020; Yan et al., 2026; 2025) and diffusion language models (Nie et al., 2026; Ye et al., 2025a; Cheng et al., 2026a; Bie et al., 2025; Wei et al., 2026), recent approaches (Chen et al., 2026; Huang et al., 2026; Cheng et al., 2026b) employ diffusion drafters with bidirectional attention to generate a draft block in a single forward pass, reducing sequential drafting overhead while achieving higher acceptance rates. Meanwhile, self-speculative methods (Fu et al., 2026; Liu et al., 2026b; Zhang et al., 2026b) further unify diffusion drafting and autoregressive verification within a shared model, enabling stronger drafting capability and longer acceptance lengths at increased drafting cost. Our work focuses on the runtime execution and efficiency of this paradigm.

Runtime Orchestration for Speculative Decoding. Beyond drafter design, recent work has increasingly optimized speculative decoding through parallel execution and adaptive runtime strategies (Zhang et al., 2026c; Lin et al., 2026), aiming to break the sequential draft-then-verify dependency and accommodate diverse serving conditions. PEARL (Liu et al., 2025) overlaps drafting and verification while adapting draft length to mitigate mutual waiting between a separate drafter and verifier. SSD (Kumar et al., 2026) further predicts likely verification outcomes before verification completes and asynchronously prepares future speculation. In contrast, our proposed RecGuide targets self-speculation where both stages invoke the same backbone, allowing future drafts to be directly co-batched with ongoing verification without model switching or additional devices. This turns the efficiency bottleneck into a limited-compute allocation problem, especially at high concurrency. At the drafting stage, prior work (Sadhukhan et al., 2025; Ning et al., 2026; Zhang et al., 2026a) explores more flexible strategies to dynamically determine the amount and configuration of speculation for higher throughput. At the verification stage, another line of work (Liu et al., 2026c; Cheng et al., 2026b; Liu et al., 2026a) selectively allocates verification effort based on confidence signals or hardware-aware runtime models. These two stages, however, are typically optimized separately. RecGuide instead treats drafting and verification as mutually informative and coupled stages, using bidirectional signals to jointly orchestrate their budgets across concurrency regimes.

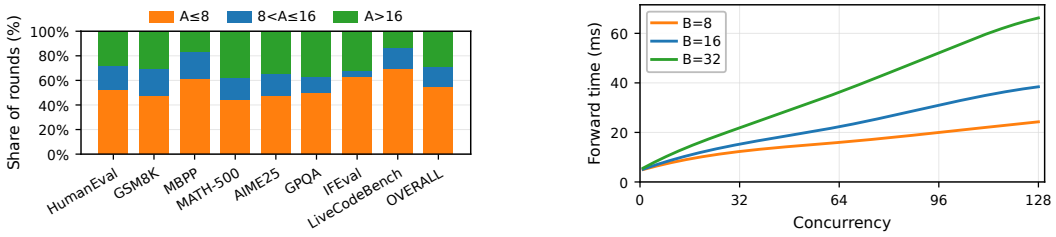
3 PRELIMINARY AND BOTTLENECK

3.1 SELF-SPECULATION WITH A SHARED BACKBONE

Unlike conventional speculative decoding, which employs a lightweight drafter alongside the target model, shared-backbone self-speculation jointly optimizes autoregressive and diffusion objectives so that a single backbone supports both diffusion drafting and autoregressive verification, as illustrated in Figure 1(b). Given a generated prefix, the model first predicts multiple future tokens in parallel under the diffusion mode and then switches to the autoregressive mode for causal verification following standard speculative decoding, accepting the correct prefix up to the first mismatch.

The two modes can be switched simply by changing the attention pattern. Let the current prefix length be P and the draft block size be B. We denote prefix positions by $\mathcal{P} = 1, \ldots, P$ and draft positions by $\mathcal{D} = P+1, \ldots, P+B$. Under diffusion drafting, the prefix preserves causal dependency, while tokens in $\mathcal{D}$ can attend to the prefix and entire draft region, enabling parallel prediction through bidirectional attention. Autoregressive verification instead enforces strict causal attention, where position i can only attend to positions $j \le i$. The corresponding attention masks are as follows:

$$M_{ij}^{\text{Diff}} = \begin{cases} 0, & i \in \mathcal{P},\ j \in \mathcal{P},\ j \le i, \\ 0, & i \in \mathcal{D},\ j \in \mathcal{P} \cup \mathcal{D}, \\ -\infty, & \text{otherwise}, \end{cases} \qquad M_{ij}^{\text{AR}} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases} \tag{1}$$



(a) (b)

Figure 2: Acceptance variability and block-dependent forward cost of self-speculation. (a) Acceptance length distribution with $B = 32$. (b) Forward time across concurrency levels for $B \in \{8, 16, 32\}$.

Consequently, M_{ij}^{Diff} enables bidirectional interaction within the draft region, whereas M_{ij}^{AR} restores left-to-right dependency. This switching allows the full backbone itself to serve as the drafter without deploying a separate draft model. However, sharing the backbone also directly couples drafting and verification at runtime, forcing the two full-backbone stages to execute sequentially within each speculative round and thereby creating efficiency characteristics distinct from conventional speculative decoding. We next examine the resulting bottlen ecks under different serving conditions.

3.2 Bottlenecks across Serving Loads

To characterize the efficiency of self-speculation across serving loads, we profile Nemotron-Labs-Diffusion 8B in terms of acceptance behavior and forward cost. As shown in Figure 2(a), with block size 32, we report the fraction of rounds whose acceptance length A falls into $A \leq 8$, $8 < A \leq 16$, and $A > 16$ across benchmarks. Nearly 30% of the rounds accept more than 16 tokens, revealing strong potential for long drafts, while over half fail within the first 8 positions, indicating substantial variation in useful drafting length. We further examine forward time under fixed block sizes of 8, 16, and 32 as concurrency increases on a single B200 GPU, as illustrated in Figure 2(b). Forward time grows with concurrency, with larger block sizes incurring increasingly higher latency under heavier workloads. Together, these profiles reveal a mismatch between highly variable acceptance potential and load-dependent drafting cost across concurrency regimes.

Low-concurrency Inefficiency. At low concurrency, inefficiency comes from both pipeline serialization and wasted drafting. Drafting and verification execute sequentially, causing mutual waiting that is often negligible with lightweight drafters but costly when both stages invoke the full backbone. Moreover, Figure 2(a) shows that large blocks can waste substantial drafting computation when mismatches occur early. Fortunately, execution remains largely memory-bound at low concurrency, leaving spare compute that can be used to overlap drafting with verification and proactively correct likely mismatches.

High-concurrency Inefficiency. As concurrency increases, Figure 2(b) shows that block size becomes increasingly constrained by compute load, favoring smaller blocks. However, uniformly using a small block remains suboptimal, since it sacrifices the long-acceptance potential observed in Figure 2(a). Since GPU latency typically grows sublinearly with the number of processed tokens, additional drafting remains beneficial when the acceptance gain outweighs the corresponding latency overhead. This motivates dynamically selecting the block size for each request rather than using a fixed global budget. The resulting divergence in per-request computation lengths will be handled through bucketed serving, as discussed in Section 5.3.

4 Reciprocal Predictability between Drafting and Verification

The bottlenecks above show that runtime orchestration must account for both varying serving loads and the dynamic nature of decoding. Shared-backbone execution exposes complementary signals from drafting and verification that can guide such adaptation. We next establish this **reciprocal predictability** and its implications for runtime decisions.

4.1 Draft Distributions Predict Verification Failures

We first examine whether the draft distribution contains predictive cues about subsequent verification. Under greedy decoding, we define the confidence of the drafted token at position i as $C_i = P_{1\text{st}}(i)$. Figure 3 compares the cumulative distribution functions (CDFs) of C_i at verifier-passed and first-rejected positions across math, code, and QA benchmarks. Passed tokens are heavily concentrated at high confidence values, whereas first-rejected positions exhibit a substantially broader distribution shifted toward lower confidence. This clear separation reveals that draft confidence can anticipate likely verification failures before the verifier is executed.

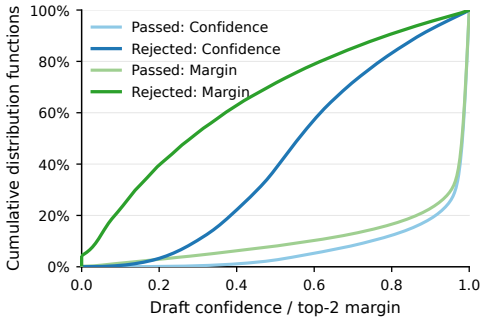


Figure 3: Cumulative distribution functions (CDFs) of draft confidence and top-2 probability margin during drafting rounds.

We further consider the probability margin between the two most likely draft candidates, $M_i = P_{1\text{st}}(i) - P_{2\text{nd}}(i)$. Its CDF exhibits a similar separation between passed and first-rejected positions. A small margin not only reflects uncertainty in the submitted draft token, but also shows that the second-ranked candidate remains competitive. Although this candidate is not necessarily correct, it offers a natural alternative at likely mismatch positions. These observations imply that likely verification failures can be anticipated and prepared for before causal verification completes.

4.2 Verification History Predicts Future Drafting Utility

Conversely, we examine whether completed verification is predictive of how far the next round should draft. Without introducing a specific utility metric, we analyze the temporal correlation of acceptance lengths across adjacent rounds, as shown in Table 1. To expose the model's full acceptance potential, the block size is fixed at 32, and the acceptance length at round t is denoted by A_t. We divide A_t into three intervals: short acceptance with $A_t \leq 8$, medium acceptance with $8 < A_t \leq 16$, and long acceptance with $A_t > 16$. These intervals align with the candidate block sizes 8, 16, and 32. Table 1 reports the conditional distribution of A_{t+1} over the three intervals. Both short and long acceptance exhibit clear temporal persistence, as the next round is most likely to remain in the same interval. This pattern suggests that local generation difficulty tends to persist across adjacent rounds, making verification history a useful prior for future drafting budgets.

Table 1: Transition probabilities of acceptance intervals between adjacent decoding rounds.

$A_t \backslash A_{t+1}$	$[1, 8]$	$(8, 16]$	$(16, 32]$
$[1, 8]$	**70.30%**	18.11%	11.60%
$(8, 16]$	58.39%	**22.42%**	19.19%
$(16, 32]$	20.83%	12.51%	**66.66%**

This persistence, however, is not static. Table 1 also shows frequent transitions between adjacent rounds. In particular, only 22.42% of medium acceptance rounds remain in the same interval, primarily because short acceptance rounds dominate all decoding rounds. Thus, each response exhibits both local persistence and regime transitions. A fixed block size therefore cannot consistently match the local drafting utility, calling for adaptive block size allocation under limited compute.

5 RecGuide: Runtime Draft-Verify Orchestration

RecGuide converts reciprocal predictability into runtime decisions, as illustrated in Figure 4. Under light workloads, spare compute prepares likely future drafts alongside verification. Once compute becomes limiting, verification history instead guides each request's drafting budget.

5.1 Predictive Draft-Verify Overlap

As discussed before, the probability margin $M_i = P_{1\text{st}}(i) - P_{2\text{nd}}(i)$ reflects draft uncertainty and second-ranked token competitiveness. We mark positions with $M_i < \delta$ as potential mismatches

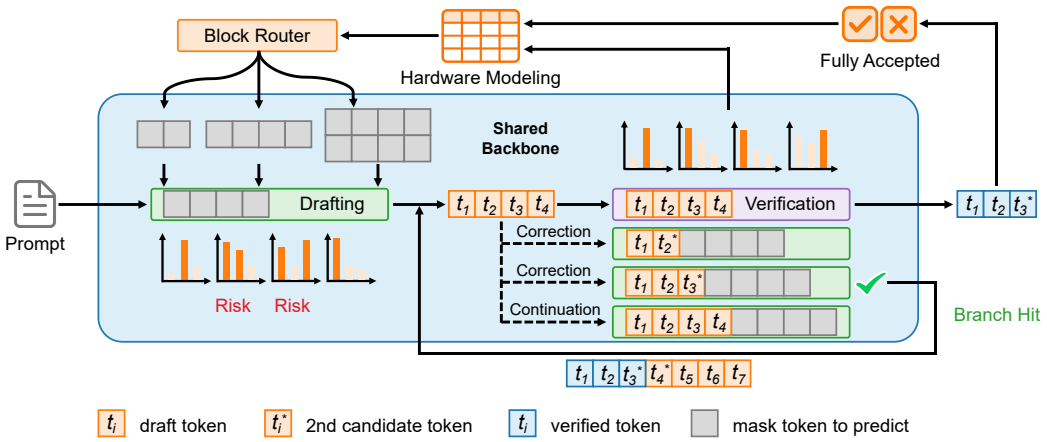


Figure 4: Overview of RecGuide runtime orchestration. Draft uncertainty guides predictive draft-verify overlap through correction and continuation branches, while verification feedback and hardware profiles guide request-specific block size allocation.

and forms a correction branch by replacing the draft token with its second candidate. Because later positions in a parallel draft are generally less reliable and less valuable for correcting the first mismatch, RecGuide retains only the earliest two risky positions. When at most two such positions exist, it additionally prepares a continuation branch assuming full acceptance of the current block. Together with the verifier, each round contains at most four logical rows, bounding extra computation.

These branches prepare drafts for the next round in parallel with current verification. A correction branch is reused only if the verifier rejects at the predicted position and produces the same replacement token, whereas retaining the continuation branch requires full acceptance of the current block and a match between its first token and the verifier bonus token. Unmatched branches are discarded. A confirmed draft can immediately repeat the procedure in the next round, enabling recursive overlap.

5.2 Verification Guided Adaptive Drafting

At higher concurrency, costly prospective branches shift the bottleneck to balancing computation and acceptance length. RecGuide uses completed verification history to select each request's next block size $B_t \in \{8, 16, 32\}$. Let e_t average verifier entropy over causally valid positions, excluding those after the first mismatch. We combine two-round certainty with the previous block outcome z_t:

$$s_t = -\frac{e_{t-1} + e_{t-2}}{2}, \qquad z_t = (B_{t-1}, F_{t-1}), \tag{2}$$

where F_{t-1} indicates whether the previous block was fully accepted. While s_t mainly reflects whether a long block should shrink under high uncertainty, z_t further encourages a larger drafting budget after full acceptance, since the true acceptance potential may extend beyond the current block boundary. Given this, RecGuide estimates the expected accepted progress $V_B(s_t, z_t)$ from calibration data for each candidate B. Following the hardware-aware cost modeling of DSpark, an offline profile provides the execution cost $T_C(B)$ under serving load C. RecGuide then selects

$$B_t^* = \arg \max_{B \in \{8,16,32\}} \left[V_B(s_t, z_t) - \rho_C T_C(B) \right], \tag{3}$$

where ρ_C weights the profiled execution cost at load C. A larger block is chosen only when its expected acceptance gain justifies the additional computation. The decision is resolved through lightweight table lookup at runtime, introducing negligible overhead.

5.3 System Implementation

Algorithmic optimization alone does not directly translate into end-to-end gains. For draft-verify overlap, naively batching the verifier and prospective branches causes redundant prefix KV reads and padding due to unequal sequence lengths. We therefore extend SGLang with shared-prefix KV access and padding-free execution, allowing all branches to reuse the canonical prefix while processing only valid query tokens. We also redesign the mapping of mixed attention workloads onto GPU SMs.

Dynamic block sizing faces a similar issue: mixing different block sizes within a batch forces shorter requests to pad to the largest block, offsetting their computational savings. RecGuide therefore adopts block-size bucketed serving. After each verification round, requests are grouped by their planned block size, and the scheduler forms batches only from requests in the same bucket, preserving the efficiency of adaptive block sizing while keeping request states and canonical KV independent.

6 Experiments

6.1 Experimental Setup

Models and Benchmarks. We evaluate RecGuide on Nemotron-Labs-Diffusion (NLD) 8B and 14B (Fu et al., 2026), a representative state-of-the-art diffusion–AR self-speculation family. Fixing the backbone enables fair decoding-paradigm comparisons with LinearSS and its quadratic variant TiDAR (Liu et al., 2026b), isolating runtime gains from architectural differences. Our evaluation covers eight benchmarks spanning three categories: **Math** (GSM8K (Cobbe et al., 2021), MATH-500 (Lightman et al., 2024), AIME25 (Zhang & Math-AI, 2025)); **Code** (HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021), LiveCodeBench (Jain et al., 2025)); **QA & instruction following** (GPQA (Rein et al., 2023), IFEval (Zhou et al., 2023)). We compare against vanilla LinearSS, DFlash, and TiDAR. Due to training budget constraints, DFlash is trained on ShareGPT (RyokoAI, 2023) following the SpecForge recipe (Li et al., 2026a), while other configurations follow official settings.

Evaluation Settings and Metrics. All experiments are conducted on a single NVIDIA B200 GPU with SGLang (Zheng et al., 2024) engine and FlashInfer (Ye et al., 2025b) backend. We evaluate our two RecGuide strategies under their corresponding serving regimes. Following NLD's recipe, all methods use greedy decoding. We set concurrency $C = 1$ and block size $B = 16$ for low-concurrency experiments. Potential mismatch positions are identified by $M_i < \delta$, with $\delta = 0.5$ throughout all experiments. For higher-concurrency serving, we vary concurrency over $C \in \{2, 4, 8, 16, 32, 64, 128\}$ and compare RecGuide against fixed block sizes $B \in \{8, 16, 32\}$ using both aggregate and per-request throughput.

6.2 Low-Concurrency Performance

Table 2 compares RecGuide with representative diffusion-AR speculative decoding paradigms under the single-request setting, reporting average acceptance length τ and end-to-end speedup over autoregressive decoding. Although LinearSS accepts substantially more tokens per round than DFlash, its two full-backbone forwards limit end-to-end speedup, allowing DFlash with a lightweight drafter to remain faster. RecGuide, by contrast, achieves the highest speedup, averaging $4.06\times$ on NLD-8B and $4.40\times$ on NLD-14B over autoregressive decoding, and delivering up to $1.8\times$ speedup over vanilla LinearSS on AIME25. Under the same two-forward accounting, RecGuide raises the effective acceptance length to more than 14 tokens, approaching the $B = 16$ limit of serial execution.

Longer acceptance, however, does not necessarily imply faster decoding even after removing the two-forward dependency of linear self-speculation, as TiDAR illustrates. Its quadratic speculative layout prepares a future draft for every possible acceptance position, incurring $O(B^2)$ speculative computation. Consequently, its longer acceptance does not translate into higher end-to-end speedup because of the increased forward cost, leaving TiDAR behind LinearSS on average. Moreover, TiDAR drafts from hypothesized acceptance positions without explicitly supplying a correction token at likely mismatches. The repeated draft may still be incorrect; even after a successful correction, later tokens in the same parallel draft cannot condition on it, potentially degrading the draft quality.

RecGuide instead assigns the second-ranked candidate to selected risky positions, providing an explicit corrected prefix once verified. Despite using far fewer prospective branches than TiDAR,

Table 2: Average acceptance length (τ) per two-forward round and decoding speedup over autoregressive decoding on NLD-8B/14B at $C = 1$, $B = 16$. RecGuide can verify two blocks in this window.

Model	Method	Metric	MATH			CODE			QA & IF		OVERALL
			GSM8K	MATH-500	AIME25	HumanEval	MBPP	LCB	GPQA	IFEval	*Avg.*
NLD-8B	LinearSS	τ	9.94	11.21	10.49	9.71	8.15	9.78	10.62	9.57	9.93
		Speedup	3.03×	2.95×	2.42×	2.91×	2.67×	2.36×	2.58×	2.53×	2.68×
	DFlash	τ	4.26	4.31	4.90	4.28	3.78	4.64	5.11	3.90	4.40
		Speedup	2.80×	2.87×	3.13×	2.83×	2.47×	3.09×	3.39×	2.57×	2.89×
	TiDAR	τ	13.44	15.80	14.54	12.92	9.18	13.48	15.18	12.52	13.38
		Speedup	2.62×	2.87×	2.34×	2.52×	1.88×	2.32×	2.57×	2.26×	2.42×
	RecGuide	τ	13.95	16.08	15.68	13.64	11.08	13.54	14.84	13.33	14.02
		Speedup	4.05×	4.66×	4.38×	4.07×	3.22×	3.91×	4.27×	3.94×	4.06×
NLD-14B	LinearSS	τ	9.74	10.50	9.35	11.99	10.81	9.27	10.79	8.35	10.10
		Speedup	2.84×	3.06×	2.36×	4.15×	3.41×	2.50×	2.82×	2.43×	2.95×
	DFlash	τ	4.78	4.37	3.81	4.93	5.49	4.14	5.28	3.80	4.58
		Speedup	3.13×	2.94×	2.50×	3.30×	3.43×	2.80×	3.54×	2.57×	3.03×
	TiDAR	τ	13.10	16.24	13.90	17.72	15.80	13.26	17.24	12.20	14.94
		Speedup	2.74×	3.09×	2.47×	3.78×	3.13×	2.48×	3.08×	2.36×	2.89×
	RecGuide	τ	13.61	15.43	13.41	18.19	12.56	13.27	16.21	10.97	14.21
		Speedup	4.22×	4.82×	4.21×	5.49×	3.74×	4.17×	5.03×	3.55×	4.40×

Table 3: Branch outcomes in draft-verify overlap. Correction and Continuation report coverage / reuse rate (%); No Hit denotes rounds unmatched by any prospective branch.

Model	Branch Outcome	MATH	CODE	QA & IF	OVERALL
NLD-8B	Correction	32.46 / 21.46	41.63 / 25.81	33.31 / 17.25	36.11 / 22.04
	Continuation	43.85 / 42.92	27.89 / 27.14	48.55 / 47.88	39.04 / 38.24
	No Hit	23.68	30.48	18.14	24.85
NLD-14B	Correction	32.37 / 21.98	30.99 / 20.38	35.44 / 18.61	32.62 / 20.54
	Continuation	37.12 / 35.95	43.81 / 36.53	39.22 / 38.88	40.15 / 36.90
	No Hit	30.51	25.20	25.33	27.22

RecGuide retains high mismatch coverage, as shown in Table 3. Correction and continuation branches cover most decoding rounds, leaving only 24.85% and 27.22% of rounds on NLD-8B and NLD-14B, respectively, unmatched by any branch. Furthermore, among matched correction branches, the second-ranked candidate matches the verifier correction in over 60% of cases, supporting explicit correction as a reliable drafting signal. Continuation branches are even more reusable: after full-block acceptance, the first draft token almost always matches the additional next token produced by the verifier. RecGuide therefore preserves acceptance lengths comparable to TiDAR with far less speculative computation, ultimately achieving the highest decoding efficiency.

6.3 HIGH-CONCURRENCY PERFORMANCE

Figure 5 evaluates RecGuide under higher serving loads, showing the trade-off between aggregate and per-request throughput across $C \in \{2, 4, 8, 16, 32, 64, 128\}$. Increasing concurrency raises aggregate throughput at the expense of per-request throughput. Compared with fixed $B \in \{8, 16, 32\}$, RecGuide favors larger blocks when compute remains available and shifts toward smaller blocks as execution becomes compute limited. This adaptive allocation consistently improves aggregate throughput on both NLD-8B and NLD-14B, advancing the overall serving frontier.

Table 4 makes this load adaptation explicit. On NLD-8B, the fraction of $B = 32$ decreases from 88.13% at $C = 2$ to 9.75% at $C = 128$, while $B = 8$ increases from 9.33% to 78.21%; NLD-14B exhibits the same transition. This confirms that RecGuide adjusts drafting budgets with serving load rather than relying on a single fixed configuration. Block-size distributions also vary across task categories at the same concurrency. In particular, QA & IF shifts toward smaller blocks earlier than MATH and CODE, consistent with higher verification uncertainty and demonstrating request-level adaptation beyond load awareness alone.

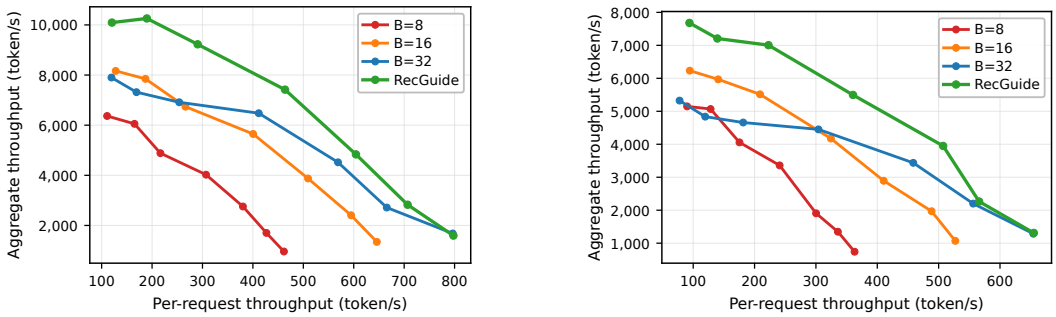


Figure 5: Aggregate and per-request throughput trade-offs under different concurrency levels on (a) NLD-8B and (b) NLD-14B. We compare fixed block sizes $B \in \{8, 16, 32\}$ with RecGuide.

Table 4: Dynamic block-size distributions across serving loads. Each entry reports the percentage of requests selecting $B = 8/16/32$, respectively; Overall is the macro average over eight benchmarks.

Model	Concurrency	MATH	CODE	QA & IF	OVERALL
NLD-8B	2	3.61 / 1.28 / 95.11	4.63 / 1.51 / 93.86	24.97 / 5.97 / 69.07	9.33 / 2.54 / 88.13
	8	0.63 / 26.91 / 72.46	0.66 / 39.18 / 60.16	12.85 / 48.73 / 38.43	3.70 / 36.96 / 59.34
	32	28.83 / 52.95 / 18.22	40.45 / 47.70 / 11.86	60.76 / 22.35 / 16.89	41.17 / 43.33 / 15.50
	128	72.46 / 17.25 / 10.30	82.03 / 11.11 / 6.87	81.12 / 5.65 / 13.23	78.21 / 12.04 / 9.75
NLD-14B	2	1.40 / 3.11 / 95.49	0.32 / 1.92 / 97.76	12.14 / 12.83 / 75.03	3.68 / 5.09 / 91.23
	8	0.56 / 42.26 / 57.18	0.04 / 37.14 / 62.82	6.27 / 58.77 / 34.97	1.79 / 44.47 / 53.74
	32	28.31 / 63.75 / 7.94	20.26 / 69.51 / 10.23	56.77 / 31.81 / 11.43	32.40 / 57.92 / 9.67
	128	84.07 / 11.56 / 4.37	83.80 / 10.21 / 5.99	83.41 / 5.94 / 10.66	83.81 / 9.65 / 6.55

These distributions also explain the per-request throughput trends in Figure 5. At $C = 2$ and $C = 128$, RecGuide predominantly selects $B = 32$ and $B = 8$, respectively, so its per-request throughput approaches the corresponding fixed-block baselines, whereas the more diverse assignments at intermediate concurrency provide greater gains from request-specific adaptation. Notably, fixed $B = 8$ remains inferior to $B = 16$ in aggregate throughput at the highest concurrency, since shorter blocks require more decoding rounds and amplify progress imbalance across concurrent requests. RecGuide avoids this drawback by selecting short blocks only when beneficial and regrouping requests by their planned block size at each round, allowing aggressive use of $B = 8$ without inheriting the inefficiency of a globally fixed short-block policy.

6.4 ABLATION STUDY

We further decompose RecGuide by progressively adding the key components of Predictive Draft-Verify Overlap and Verification Guided Adaptive Drafting to vanilla LinearSS under their respective load regimes. We report aggregate throughput in tokens per second (TPS) on NLD-8B / 14B.

Predictive Draft-Verify Overlap. Table 5 ablates the prospective branches at $C = 1$. Starting with the first correction branch, we add the second correction and continuation branches to isolate the benefits of broader mismatch coverage and full-block continuation. The first correction branch alone improves throughput by 34.29% and 28.72% on NLD-8B / 14B, respectively. The second correction branch yields a smaller gain, which we attribute to later risky positions being less causally grounded and harder to predict, supporting our decision to bound the number of correction branches. Adding the con-

Table 5: Ablation of prospective branches in Predictive Draft-Verify Overlap at $C = 1$. TPS is averaged over all benchmarks.

Method	NLD-8B	NLD-14B
LinearSS	595	470
+ 1st Correction	799 (+34.29%)	605 (+28.72%)
+ 2nd Correction	811 (+36.30%)	612 (+30.21%)
+ Continuation	903 (+51.76%)	702 (+49.36%)

tinuation branch further raises the improvement to 51.76% and 49.36%. These results show that three prospective branches capture most of the overlap benefit while keeping computation bounded.

Verification Guided Adaptive Drafting. We next ablate the adaptive block policy over $C \in \{2, 4, 8, 16, 32, 64, 128\}$, as shown in Table 6. Fixed Block follows NLD's official recipe, using a concurrency-specific block size shared by all requests. Adding two-round entropy s_t improves TPS by 10.49% / 7.60% on NLD-8B / 14B. The previous block size and full-acceptance state z_t further guide budget expansion. Request-specific block sizes introduce unequal draft lengths within batches, so padding to the largest block can offset these computational savings. Block-size bucketing groups requests by their selected size, translating compute savings into serving throughput gains. Together, adaptive selection and bucketed execution improve TPS by 39.84% and 32.80%, respectively.

Table 6: Ablation of verification guided adaptive drafting. TPS is averaged over all benchmarks and concurrency levels.

Method	NLD-8B	NLD-14B
Fixed Block	4725	3756
+ s_t	5221 (+10.49%)	4041 (+7.60%)
+ z_t	5254 (+11.19%)	4071 (+8.40%)
+ Bucketing	6608 (+39.84%)	4987 (+32.80%)

7 Conclusion

This work presents Reciprocal Guidance (RecGuide), a runtime draft-verify orchestration framework for diffusion-AR self-speculation. RecGuide leverages reciprocal predictability between drafting and verification to overlap the two stages at low concurrency and adapt request-specific draft budgets under heavier workloads. Across a wide range of concurrency levels, RecGuide consistently improves throughput over vanilla LinearSS, achieving up to 1.8× speedup. These results highlight self-speculation that reuses the full target backbone as the drafter as a promising alternative for efficient speculative decoding across diverse serving loads and concurrency regimes.

References

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL https://arxiv.org/abs/2108.07732.

Tiwei Bie, Maosong Cao, Kun Chen, Lun Du, Mingliang Gong, Zhuochen Gong, Yanmei Gu, Jiaqi Hu, Zenan Huang, Zhenzhong Lan, Chengxi Li, Chongxuan Li, Jianguo Li, Zehuan Li, Huabin Liu, Ling Liu, Guoshan Lu, Xiaocheng Lu, Yuxin Ma, Jianfeng Tan, Lanning Wei, Ji-Rong Wen, Yipeng Xing, Xiaolu Zhang, Junbo Zhao, Da Zheng, Jun Zhou, Junlin Zhou, Zhanchao Zhou, Liwang Zhu, and Yihong Zhuang. Llada2.0: Scaling up diffusion language models to 100b, 2025. URL https://arxiv.org/abs/2512.15745.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 5209–5235. PMLR, 2024. URL https://proceedings.mlr.press/v235/cai24b.html.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling, 2023. URL https://arxiv.org/abs/2302.01318.

Jian Chen, Yesheng Liang, and Zhijian Liu. Dflash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher

Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. URL https://arxiv.org/abs/2107.03374.

Zhuoming Chen, Avner May, Ruslan Svirschevski, Yuhsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. Sequoia: Scalable, robust, and hardware-aware speculative decoding. *arXiv preprint arXiv:2402.12374*, 2024.

Shuang Cheng, Yihan Bian, Dawei Liu, Yuhua Jiang, Yihao Liu, Linfeng Zhang, Qian Yao, Zhongbo Tian, Wenhai Wang, Qipeng Guo, et al. Sdar: A synergistic diffusion-autoregression paradigm for scalable sequence generation. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 22058–22075, 2026a.

Xin Cheng, Xingkai Yu, Chenze Shao, Jiashi Li, Yunfan Xiong, Yi Qian, Jiaqi Zhu, Shirong Ma, Xiaokang Zhang, Jiasheng Ye, et al. Dspark: Confidence-scheduled speculative decoding with semi-autoregressive generation. *arXiv preprint arXiv:2607.05147*, 2026b.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL https://arxiv.org/abs/2110.14168.

Yonggan Fu, Lexington Whalen, Abhinav Garg, Chengyue Wu, Maksim Khadkevich, Nicolai Oswald, Enze Xie, Daniel Egert, Sharath Turuvekere Sreenivas, Shizhe Diao, Chenhan Yu, Ye Yu, Weijia Chen, Sajad Norouzi, Jingyu Liu, Shiyi Lan, Ligeng Zhu, Jin Wang, Jindong Jiang, Morteza Mardani, Mehran Maghoumi, Song Han, Ante Jukic, Nima Tajbakhsh, Jan Kautz, and Pavlo Molchanov. Nemotron-labs-diffusion: A tri-mode language model unifying autoregressive, diffusion, and self-speculation decoding. Technical report, NVIDIA, 2026.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

Jianuo Huang, Yaojie Zhang, Qituan Zhang, Hao Lin, Hanlin Xu, and Linfeng Zhang. Domino: Decoupling causal modeling from autoregressive drafting in speculative decoding. *arXiv preprint arXiv:2605.29707*, 2026. doi: 10.48550/arXiv.2605.29707. URL https://arxiv.org/abs/2605.29707.

Naman Jain, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, volume 2025, pp. 58791–58831, 2025.

Tanishq Kumar, Tri Dao, and Avner May. Speculative speculative decoding, 2026. URL https://arxiv.org/abs/2603.03251.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International conference on machine learning*, pp. 19274–19286. PMLR, 2023.

Shenggui Li, Chao Wang, Yikai Zhu, Yubo Wang, Fan Yin, Shuai Shi, Yefei Chen, Xiaomin Dong, Qiaoling Chen, Jin Pan, Ji Li, Laixin Xie, Yineng Zhang, Lei Yu, Yonggang Wen, Ivor Tsang, and Tianwei Zhang. Specforge: A flexible and efficient open-source training framework for speculative decoding, 2026a. URL https://arxiv.org/abs/2603.18567.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, pp. 7421–7432, 2024a.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning*, pp. 28935–28948. PMLR, 2024b. URL https://proceedings.mlr.press/v235/li24bt.html.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-3: Scaling up inference acceleration of large language models via training-time test. *Advances in Neural Information Processing Systems*, 38:136737–136756, 2026b.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *International Conference on Learning Representations*, volume 2024, pp. 39578–39601, 2024.

Luxi Lin, Zhanpeng Zeng, Shuang Peng, Songwei Liu, and Rongrong Ji. Retrace: Rejected-trajectory conditioning for speculative decoding, 2026. URL https://arxiv.org/abs/2608.29748.

Hong Liu, Rui Cen, Junhan Shi, Guangshuo Qin, Jiebin Zhang, Tianyu Liu, Runzhi Fan, Guoliang Zhao, Ruobing Xie, Kai Zhang, Song Liu, Guanghua Yu, and Jianchen Zhu. Angelspec: Towards real-world high performance inference with speculative decoding, 2026a. URL https://arxiv.org/abs/2607.25852.

Jingyu Liu, Xin Dong, Zhifan Ye, Rishabh Mehta, Yonggan Fu, Vartika Singh, Ce Zhang, and Pavlo Molchanov. Tidar: Think in diffusion, talk in autoregression. *Proceedings of Machine Learning and Systems*, 8:748–762, 2026b.

Tianyu Liu, Yun Li, Qitan Lv, Kai Liu, Jianchen Zhu, Winston Hu, and Xiao Sun. Pearl: Parallel speculative decoding with adaptive draft length. In *International Conference on Learning Representations*, volume 2025, pp. 1085–1104, 2025.

Tianyu Liu, Yuhao Shen, Rui Cen, Junhan Shi, Jiebin Zhang, Guangshuo Qin, Hong Liu, Song Liu, Guanghua Yu, and Jianchen Zhu. D-cut: Adaptive verification depth pruning for batched speculative decoding, 2026c. URL https://arxiv.org/abs/2607.14647.

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *Advances in Neural Information Processing Systems*, 38:50608–50646, 2026.

Zhiyuan Ning, Jiawei Shao, Ruge Xu, Xinfei Guo, Jun Zhang, Chi Zhang, and Xuelong Li. Cas-spec: Cascade adaptive self-speculative decoding for on-the-fly lossless inference acceleration of llms. *Advances in Neural Information Processing Systems*, 38:65145–65167, 2026.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark, 2023. URL https://arxiv.org/abs/2311.12022.

RyokoAI. Sharegpt52k: A collection of chatgpt conversations. https://huggingface.co/datasets/RyokoAI/ShareGPT52K, 2023.

Ranajoy Sadhukhan, Jian Chen, Zhuoming Chen, Vashisth Tiwari, Ruihang Lai, Jinyuan Shi, Ian Yen, Avner May, Tianqi Chen, and Beidi Chen. Magicdec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding. In *International Conference on Learning Representations*, volume 2025, pp. 6835–6850, 2025.

Linye Wei, Wenjue Chen, Pingzhi Tang, Xiaotian Guo, Le Ye, Runsheng Wang, and Meng Li. Orchestrating dual-boundaries: An arithmetic intensity inspired acceleration framework for diffusion language models, 2026. URL https://arxiv.org/abs/2511.21759.

RenYe Yan, Jikang Cheng, Yaozhong Gan, Shikun Sun, You Wu, Yunfan Yang, Liang Ling, Jinlong Lin, Yeshuang Zhu, Jie Zhou, et al. Entropy-adaptive diffusion policy optimization with dynamic step alignment. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 1924–1934, 2025.

Renye Yan, Jikang Cheng, Shikun Sun, Yi Sun, You Wu, Wei Peng, Zongwei Wang, Ling Liang, Junliang Xing, and Yimao Cai. Do less, achieve more: Do we need every-step optimization for rl fine-tuning of diffusion models? *arXiv preprint arXiv:2605.15855*, 2026.

Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models, 2025a. URL https://arxiv.org/abs/2508.15487.

Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, et al. Flashinfer: Efficient and customizable attention engine for llm inference serving. *Proceedings of Machine Learning and Systems*, 7, 2025b.

Jiebin Zhang, Zhenghan Yu, Liang Wang, Nan Yang, Eugene Yu, Zheng Li, Yifan Song, Dawei Zhu, Xingxing Zhang, Furu Wei, et al. Learning to draft: Adaptive speculative decoding with reinforcement learning. In *International Conference on Learning Representations*, volume 2026, pp. 30798–30819, 2026a.

Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. Draft& verify: Lossless large language model acceleration via self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 11263–11282, 2024.

Yaojie Zhang, Jianuo Huang, Junlong Ke, Yuhang Han, Yongji Long, Tianchen Zhao, Biqing Qi, and Linfeng Zhang. Flexdraft: Flexible speculative decoding via attention tuning and bonus-guided calibration, 2026b. URL https://arxiv.org/abs/2605.20022.

Yaojie Zhang, Linfeng Zhang, Bin Cui, and Xupeng Miao. Dflow: Enabling verifier information flow in block diffusion speculative decoding, 2026c. URL https://arxiv.org/abs/2609.06498.

Yifan Zhang and Team Math-AI. American invitational mathematics examination (AIME) 2025, 2025. URL https://huggingface.co/datasets/math-ai/aime25.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.

Shuzhang Zhong, Zebin Yang, Ruihao Gong, Runsheng Wang, Ru Huang, and Meng Li. Propd: Dynamic token tree pruning and generation for llm parallel decoding. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, pp. 1–8, 2024.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models, 2023. URL https://arxiv.org/abs/2311.07911.